

Hippocampal Neuropixels Simulation and Causal Manifold Decoding

Technical project document

hippo repository

July 19, 2026

Abstract

This document specifies a Python research pipeline that (i) simulates hippocampal single-unit activity during open-field navigation, (ii) records it through a Neuropixels-like probe with sorting degradation, and (iii) decodes behavioral latent variables from *causal* population spike counts. Hyperparameter search over decoder models, integration windows, and optional manifold features is performed automatically; the selected configuration is then replayed in a closed-loop causal setting. We document the neural rate equations, probe anatomy, causal decoding architecture, decoder model zoo, and manifold feature modes with explicit mathematical definitions and default numerical parameters.

Contents

1 Introduction and scientific framing

The simulator knows the true behavioral state. The decoder observes only spike times (ground-truth Poisson spikes or spikes after Neuropixels-like degradation and sorting). Comparing these two spike sources quantifies information loss due to extracellular recording and sorting. Searching causal integration windows asks how much recent neural history is required to decode spatial, proprioceptive, and contextual variables. Searching decoder families asks which estimator best extracts those latents from population activity. Closed-loop replay tests whether decoded estimates can drive realtime triggers.

Six computational operations are kept conceptually separate:

1. population partition (which units enter a model);
2. observation construction (causal spike counts);
3. manifold geometry (map to latents z_t);
4. temporal dynamics on the manifold (history of length L);
5. behavioral decoding ($\hat{y}_{t+\tau}$);
6. interregional communication (planned).

2 Public software workflow

The intended user interface consists of three scripts:

1. `run_simulation.py` — generate behavior, rates, spikes, recording, sorting;
2. `run_full_decoder_workflow.py` — compare models/windows, select the best configuration, run closed-loop replay, optionally plot;
3. `run_visualizations.py` — regenerate figures from saved outputs only (never retrains).

Listing 1: Standard end-to-end commands.

```
python run_simulation.py \  
  --output outputs/ratinabox_002 \  
  --seed 2 \  
  --neural-backend ratinabox_neurons  
  
python run_full_decoder_workflow.py \  
  --input outputs/ratinabox_002 \  
  --output outputs/ratinabox_002 \  
  --compare-sources \  
  --closed-loop-target spatial_context \  
  --selection-policy shortest_near_optimal \  
  --compile-pdf  
  
python run_visualizations.py \  
  --experiment outputs/ratinabox_002 \  
  --all --compile-pdf
```

Hyperparameter tuning is intentionally *internal* to the decoder workflow: users need not manually choose W or call separate comparison/replay scripts for the standard path.

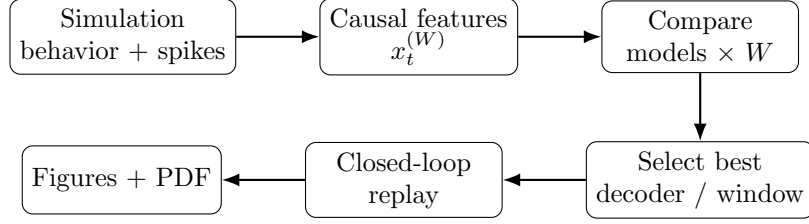


Figure 1: High-level pipeline. Tuning and selection occur under the hood before realtime replay.

3 Simulation pipeline

Default session duration is 600 s (10 minutes). Behavior and rate updates run at 20 Hz ($\Delta t = 0.050$ s).

1. **Behavior:** RatInABox open-field trajectory (thigmotaxis, stalls, smooth turns).
2. **Features:** place, head direction, speed, acceleration, boundary distance, theta phase, ripple.
3. **Rates:** custom CA1/CA2/CA3/DG equations *or* RatInABox neuron classes.
4. **Spikes:** inhomogeneous Poisson sampling (ground truth).
5. **Recording:** multi-channel templates, noise, motion amplitude drift, collisions.
6. **Sorting:** Kilosort-like re-extraction with misses, jitter, contamination.

3.1 Neural backends

`custom_rate_equations` Explicit hippocampal rate model (Section ??).

`ratinbox_neurons` RatInABox place / HD / boundary / speed cells mapped onto anatomical units (recommended for quick experiments).

Primary simulation outputs: `behavior.csv`, `units.csv`, `spikes_ground_truth.csv`, `spikes_sorted.csv`, `anatomy_regions.csv`, `summary.json`.

4 Neural rate model

Source of truth in code: `hippo_sim/rate_equations.py`, `hippo_sim/config.py`.

4.1 Dynamics

For each unit i , rates $R_i(t)$ obey the linear filter

$$\tau_i \frac{dR_i}{dt} = -R_i + \lambda_i^{\text{target}}(t), \quad (1)$$

integrated with Euler steps at the behavioral timestep $\Delta t = 0.05$ s.

4.2 Target drive

Let $p(t)$ be allocentric position, $\theta(t)$ head direction, $v(t)$ speed, $a(t)$ acceleration, $d_{\text{wall}}(t)$ distance to the nearest wall, $\phi_\theta(t)$ theta phase, and $r(t)$ a ripple envelope. Define

$$f_i^{\text{place}}(t) = \exp\left(-\frac{\|p(t) - p_{0,i}\|^2}{2\sigma_i^2}\right), \quad (2)$$

$$f_i^{\text{HD}}(t) = \frac{\exp(\kappa_i \cos(\theta(t) - \theta_i^{\text{pref}}))}{\exp(\kappa_i)/I_0(\kappa_i)}, \quad (3)$$

$$f_i^{\text{speed}}(t) = \frac{\max(0, v(t) - v_i^{\text{thresh}})}{30 - v_i^{\text{thresh}}}, \quad (4)$$

$$f_i^{\text{acc}}(t) = \text{clip}(|a(t)|/50, 0, 1), \quad (5)$$

$$f_i^{\text{bnd}}(t) = \exp\left(-\frac{d_{\text{wall}}(t)^2}{2 \cdot 15^2}\right), \quad (6)$$

$$f_i^\theta(t) = 1 + w_i^\theta \cos \phi_\theta(t). \quad (7)$$

The multiplicative drive is

$$\begin{aligned} \text{drive}_i(t) = & b_i + A_i f_i^{\text{place}} (1 + w_i^{\text{HD}} f_i^{\text{HD}}) (1 + w_i^{\text{speed}} f_i^{\text{speed}}) (1 + 0.2 f_i^{\text{acc}}) f_i^\theta \\ & + w_i^{\text{bnd}} A_i f_i^{\text{bnd}} + w_i^{\text{ripple}} A_i r(t). \end{aligned} \quad (8)$$

Region-specific modifications:

- **CA3:** $\text{drive}_i \leftarrow \text{drive}_i + w_i^{\text{rec}} \bar{R}(t)$.
- **DG sparsity:** if $f_i^{\text{place}} (1 + w_i^{\text{speed}} f_i^{\text{speed}}) < \theta_i^{\text{sparse}}$, then $\text{drive}_i \leftarrow 0.1 b_i$.

Finally

$$\lambda_i^{\text{target}}(t) = \max(\text{drive}_i(t) \xi(t) g_i(t), 0), \quad (9)$$

where $\xi(t)$ is a shared arousal/state Ornstein–Uhlenbeck (OU) factor and $g_i(t)$ is a per-unit gain OU process.

4.3 Default parameters

Table 1: Rate parameters by cell type.

Parameter	CA1_pyr	CA2_pyr	CA3_pyr	DG_granule	Units
τ	0.05	0.05	0.06	0.08	s
b (baseline)	0.5	0.4	0.3	0.05	Hz
A (amplitude)	12.0	10.0	8.0	15.0	Hz
σ_{place}	10	8	12	6	cm
w_{HD}	0.4	0.35	0.2	0.1	—
κ_{HD}	2.0	2.5	1.5	1.0	—
w_{speed}	0.3	0.25	0.2	0.5	—
v_{thresh}	2	2	2	3	cm/s
w_θ	0.25	0.15	0.1	0.05	—
w_{ripple}	0.5	0.1	0.6	0.0	—
w_{boundary}	0.2	0.15	0.1	0.3	—
$w_{\text{recurrent}}$	—	—	0.15	—	—
θ_{sparse}	—	—	—	0.3	—

Table 2: Drift, recording, and sorting defaults.

Parameter	Value	Units / notes
Place drift SD	0.1	cm/min
Place update interval	30	s
State OU τ, σ	120, 0.15	s, —
Gain OU τ, σ	180, 0.1	s, —
Channels / pitch	384 / 20	— / μm
Sample rate	30	kHz
Template span	3–10	channels
Amplitude range	20–200	μV
Noise SD	15	μV
Miss rate / jitter	0.12 / 0.3	— / ms
Contamination / merge	0.08 / 0.04	—

5 Probe anatomy

Neuropixels 1.0 single-shank geometry along a dorsal hippocampal depth axis.

Table 3: Region segments (approximate channel ranges; exact mapping in `anatomy_regions.csv`).

Region	Layer	Depth (μm)	Dens.	Cell type
CA1	oriens	0–200	2.0	CA1_pyr
CA1	pyramidal	200–400	8.0	CA1_pyr
CA1	radiatum	400–600	5.0	CA1_pyr
CA2	pyramidal	600–800	6.0	CA2_pyr
CA3	pyramidal	800–1400	7.0	CA3_pyr
DG	granule	1400–1800	10.0	DG_granule
DG	hilus	1800–2000	3.0	DG_granule

6 Causal decoding

6.1 Observation model

At each behavioral frame time t , the decoder builds a causal population count vector

$$x_t^{(W)} \in \mathbb{R}^N, \quad (x_t^{(W)})_i = \#\{\text{spikes of unit } i \text{ in } [t - W, t)\}. \quad (10)$$

The interval is half-open: spikes with time $\geq t$ are never included. Optional rate features are $r_t^{(W)} = x_t^{(W)} / W$.

6.2 Timing symbols

Default window grid:

$$W \in \{0.050, 0.100, 0.250, 0.500, 1.000\} \text{ s}. \quad (11)$$

Short W lowers latency but yields fewer spikes; long W improves count reliability at the cost of effective delay.

6.3 Architectural separation

$$x_t^{(W)} \longrightarrow z_t \longrightarrow Z_t^{(L)} = [z_{t-L+1}, \dots, z_t] \longrightarrow \hat{y}_{t+\tau}. \quad (12)$$

Table 4: Timing notation.

Symbol	Default	Meaning
t / behavior timestamps	from <code>behavior.csv</code>	Prediction grid
dt_{update}	0.050 s (20 Hz)	One prediction per frame
W	searched	Causal integration window
z_t	—	Current manifold coordinate
L	optional	Latent history length (frames)
τ	0 (searchable)	Prediction lag for $\hat{y}_{t+\tau}$
train fraction	0.70	Contiguous temporal split

Here W provides spike evidence for the *current* state; L models recent movement through the manifold; τ is a future prediction horizon. These roles must not be collapsed into a single “decoder memory” parameter.

6.4 Decoding targets

Table 5: Targets and primary selection metrics.

Target	Family	Primary metric	Direction
position	continuous (2D)	mean position error (cm)	lower
speed	continuous	R^2	higher
acceleration	continuous	R^2	higher
head_direction	circular	mean circular error (deg)	lower
distance_to_wall	continuous	R^2	higher
spatial_context	categorical	balanced accuracy	higher
movement_state	categorical	balanced accuracy	higher
wall_distance_bin	categorical	balanced accuracy	higher

7 Decoder model zoo

Implemented in `realtime/decoder_models.py`.

7.1 Quick mode (default)

- Regression: Ridge ($\alpha = 1$), PCA-Ridge (95% variance), Random Forest (100 trees, depth 12).
- Classification: Logistic Regression (balanced), Random Forest classifier (balanced).
- Bayesian place decoder when applicable (Section ??).

7.2 Full mode

Additionally includes Elastic Net ($\alpha = 0.1$, ℓ_1 -ratio 0.5), PLS (10 components), histogram gradient boosting, k -NN ($k = 15$), linear SVM, and a causally smoothed Bayesian place decoder.

7.3 Selection policies

After evaluating all (model, W , feature_mode) combinations per target:

`best_accuracy` pick the best primary metric;

`shortest_near_optimal` (default) pick the shortest W within 5% of the best score ($\leq 1.05 \times$ best if lower-is-better; $\geq 0.95 \times$ best if higher-is-better).

Closed-loop replay loads the selected serialized model and does not retrain when comparison artifacts already exist.

8 Bayesian place decoder

A Poisson likelihood place-map decoder on a $n_{\text{bins}} \times n_{\text{bins}}$ grid ($n_{\text{bins}} = 20$ by default). Occupancy is floored at 10^{-3} . Optional causal posterior smoothing uses weight $\alpha = 0.7$ on the previous posterior only (never future). Schematically,

$$\log P(\text{bin} \mid x_t) \propto \log P(\text{bin}) + \sum_i \left[x_{t,i} \log \lambda_i(\text{bin}) - \lambda_i(\text{bin}) W \right]. \quad (13)$$

A derived-context variant maps the place posterior to categorical spatial / wall-distance labels.

9 Manifold features

9.1 Static manifold-informed decoding

$$x_t^{(W)} \xrightarrow{E} z_t \xrightarrow{\text{decoder}} \hat{y}_t. \quad (14)$$

The encoder E is fit on the training split only and frozen for test / realtime.

Table 6: Static feature modes (`realtime/manifold_features.py`).

Mode	Transform	Grouping	Latent dim
<code>counts</code>	$z = x$	—	N
<code>rates</code>	$z = x/W$	—	N
<code>global_pca</code>	PCA	all units	k (default 3)
<code>region_pca</code>	PCA per region	<code>region</code>	k / group
<code>layer_pca</code>	PCA per layer	<code>layer</code>	k / group
<code>cell_type_pca</code>	PCA per type	<code>cell_type</code>	k / group
<code>rate_model_pca</code>	PCA per model	<code>rate_model</code>	k / group

Quick default modes: `counts`, `global_pca`, `region_pca`.

9.2 Optional temporal manifold decoding

Enabled with `--enable-temporal-manifold`:

$$x_t^{(W)} \rightarrow z_t \rightarrow Z_t^{(L)} = [z_{t-L+1}, \dots, z_t] \rightarrow \hat{y}_{t+\tau}. \quad (15)$$

Defaults: representations `{raw,pca}` with $\sqrt{\text{counts}}$ + standardization on the temporal PCA path; history lengths $L \in \{1, 2, 5, 10, 20\}$ frames (50 ms–1 s at 20 Hz); models `raw_static`, `static_latent`, `flattened_history`; controls: averaged and time-shuffled histories. Selection uses contiguous train/validation/test splits with a leakage gap; validation only is used for model selection. Planned extensions: causal GRU/TCN, autoencoders, CEBRA.

10 Outputs and visualization

Listing 2: Experiment directory layout.

```
outputs/<run>/
  behavior.csv, units.csv, spikes_*.csv, summary.json
  decoder_comparison/{sorted,ground_truth}/
  realtime_decoding/{spike_source}/{target}_{policy}/
  decoding/ # optional temporal W x L
  figures/
    decoder_comparison/
    realtime_decoding/
  output.pdf
```

`run_visualizations.py` detects available folders and plots simulation, comparison, and realtime figures without recomputing models. With `--compile-pdf`, all PNGs are assembled into a sectioned `figures/output.pdf`.

11 Limitations

- Simulated region functions and rate parameters are configurable hypotheses, not recovered biology.
- Sorted spikes lose some ground-truth metadata (identity, projections).
- Supervised embeddings (when added) reflect supplied labels and should not be treated as complete intrinsic geometry.
- Realtime-compatible methods must not use future spikes or future smoothing.
- Simulation results require later experimental validation.

12 Repository map (selected)

Table 7: Selected modules.

Path	Role
hippo_sim/	Behavior, rates, spikes, recording, sorting
realtime/timing.py	dt_{update} , W , L , τ
realtime/spike_features.py	Causal count matrices
realtime/decoder_models.py	Model zoo
realtime/bayesian_decoder.py	Bayesian place decoder
realtime/manifold_features.py	Static manifold modes
realtime/manifolds/	Temporal raw / PCA encoders
realtime/temporal/	History models and $W \times L$ search
realtime/workflow.py	Public decoder orchestration
visualization/	Figures and PDF compilation

License: research / educational use.

Companion README: project root README.md (Unicode equations for Markdown preview).